exec.x

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 1m 55s >

Este arquivo é malware (ou, no mínimo, um executável altamente ofuscado e suspeito, típico de malware).

Análise passo a passo

1. **Tipo de arquivo e estrutura básica**
   - É um executável ELF 64-bit (x86-64), tipo `DYN` (PIE — Position Independent Executable), compilado com GCC 14.2.0 (Debian).
   - Tamanho: 16.688 bytes.
   - Build ID: `8909868d12f6bb4d6c291132af4384ee4ea4c8c5`.
   - Depende apenas de `libc.so.6`.
   - Seções normais (`.text`, `.rodata`, `.data`, `.bss`, etc.). Não está empacotado com UPX ou similar (entropia geral ~3,94 — relativamente baixa).
2. **Strings e imports**
   - Strings legíveis relevantes:
     - `"E: neither argv[0] nor $_ works."`
     - Formatos: `"x%lx"`, `"=%lu %d"`, `"%lu %d%c"`.
     - Funções importadas: `execvp`, `putenv`, `getenv`, `getpid`, `time`, `stat`, `sprintf`, `strdup`, `malloc` / `calloc`, `memcpy` / `memset` / `memcmp`, `fprintf`, `exit`, etc.
   - Nenhuma string de ajuda, versão, uso legítimo, URL, ou mensagem de programa normal. A única mensagem de erro é a relacionada a `argv[0]` / `$_` (variável de shell).
3. **Comportamento do código (disassembly)**
   - O `main` (por volta de `0x17a3`) tenta obter o caminho do próprio executável via `argv[0]`. Se falhar, tenta `getenv("_")`. Se ambos falharem, imprime o erro e sai.
   - Chama funções que usam `getpid()`, processam dados e fazem `putenv()` (define variável de ambiente baseada no PID e outros valores).
   - Há uma rotina clara de **criptografia de fluxo** (estilo RC4) nas funções por volta de `0x12da` e `0x13cf`:
     - Usa uma S-box (em torno de `0x4920` no `.bss`/dados).
     - Índices, swaps de bytes e operações típicas de Key Scheduling Algorithm (KSA) + Pseudo-Random Generation Algorithm (PRGA).
   - Dados de alta entropia (~2 KB) na seção `.data` (começando em VA `0x40e0`, tamanho ~0x7e6) são processados por essa rotina de “descriptografia”.

- Após o processamento, o código monta um `argv` e chama **`execvp`** com um caminho/comando que estava ofuscado (os "strings" em endereços como `0x488c`, `0x4736` etc. são lixo binário até serem descriptografados em runtime).

4. **Indicadores de malícia / ofuscação**
   - Strings e caminhos de execução estão criptografados e só são revelados em tempo de execução (técnica clássica de malware para evitar detecção estática e análise).
   - Uso de `putenv` + PID sugere possível mecanismo de comunicação, anti-análise ou lock entre processos.
   - `execvp` de conteúdo descriptografado = o binário é basicamente um *dropper/loader/wrapper* que esconde o que realmente executa.
   - Ausência total de comportamento legítimo visível (sem lógica de ferramenta normal, sem mensagens úteis, sem dependências além de libc).
   - Hashes (SHA-256 `d166fe35d07e1303ee8f1b082a0c3d50b66eead61e30ef40753dd6e1fef56a30` e MD5 `e9e8ecb426a8d7a053fdb87f9982166f`) não retornam resultados públicos conhecidos de software legítimo.

Conclusão final

O arquivo **não é benigno**. Trata-se de um executável ofuscado que:

- Descriptografa dados embutidos em runtime (RC4-like),
- Manipula o ambiente (`putenv` com base em PID),
- E executa outro programa/comando via `execvp`.

Isso é o padrão clássico de loaders, droppers ou stubs de malware. Mesmo que o payload final seja "inofensivo", a ofuscação intencional + execução indireta o coloca na categoria de malware/suspicious binary.

Recomendação: Não execute. Analise apenas em sandbox isolada (com snapshot, monitoring de syscalls, rede desligada, etc.).

Qual é a categoria principal deste malware?

Worked for 36s >

Categoria principal: Loader (Encrypted Loader / Crypter stub)

Justificativa

O binário `exec.x` funciona como um **loader ofuscado**:

- Contém um bloco de dados de alta entropia (~2 KB) na seção `.data`.
- Implementa uma rotina de criptografia de fluxo (variante de RC4) para descriptografar esse bloco em tempo de execução.

- Após a descriptografia, monta argumentos e chama `execvp` para executar o conteúdo revelado.
- Manipula o ambiente (`putenv` com base no PID) e tenta localizar o próprio caminho via `argv[0]` ou `$_` .

Esse é o padrão clássico de um **stub/loader criptografado** (crypter): o arquivo em si não contém a lógica maliciosa final de forma legível — ele apenas "abre" e executa o payload embutido.

Classificações secundárias possíveis

Categoria	Adequação	Motivo
Loader	Principal	Descriptografa e executa payload embutido via `execvp`
Trojan	Alta	Classificação genérica comum em AVs para esse tipo de stub
Dropper	Média	Menos preciso (o payload já está dentro do arquivo, não é baixado)
Crypter	Alta	Técnica de ofuscação usada
Backdoor / Miner / Ransomware	Baixa	Não há evidência dessas funcionalidades no stub analisado

Resumo: A categoria principal e mais precisa para este arquivo é **Loader**.

↘ Qual é a categoria secundária

↘ Explique a técnica de criptografia RC4 usada

↘ Analisar a entropia do binário